D0 Audit, Does a Withdrawal Index Bind to One Payout Across Retries

Hilawe Semunegus, on a question raised by Joel Valenzuela (TheDesertLynx)

The phase-D0 audit for project D1 in dash-ideas-scoping.md. It answers the one question the instant-withdrawal case rests on, whether a Platform credit withdrawal keeps the same payout when it expires and is re-signed. It is a source read of two public repositories at pinned commits, dashpay/platform at `f7d7c8d` and dashpay/dash at `2bbf4a4`. No node was run. Every `path:line` citation below is as of those two commits, and line numbers will drift on later revisions, so read them against the pinned commit hashes rather than current master.

Plain words. The instant-withdrawal idea only works if a withdrawal that a wallet sees in the mempool is guaranteed to settle as the exact payment it shows. The worry was that an expired withdrawal gets retried as a different transaction that could pay a different amount, in which case crediting early would be unsafe. The code says the worry does not apply. When Platform re-signs an expired withdrawal it keeps the same index, the same output script, the same amount, and even the same fee. The only things that change are the block height it was signed at and the quorum that signed it. On the Core side, once any transaction for a given index is mined into the chain, that index can never be reused on that chain, so a withdrawal settles at most once (with ChainLock the point where that becomes permanent). Put together, a valid quorum-signed withdrawal in your own node's mempool will settle as exactly what it shows, or not yet, but never as something else. That is the strong result the design wanted, and it means instant acceptance is a policy change, not a protocol change.

Verdict

Path A. The retry-payout invariant already holds in the Platform implementation. Instant acceptance of credit withdrawals is achievable as receiving-wallet and exchange policy plus a spec write-down in Asset Locks v2, with no Core consensus change and no new network message. The audit also surfaced two real edge cases, both on the withdrawing user's side rather than the receiver's, that the specification should address while it is open.

What changes on retry, and what does not

A withdrawal is a document on Platform (`withdrawals-contract`), created when an identity requests a credit withdrawal. Its payout fields are written once at creation and never rewritten. On expiry and rebroadcast only the signing-context fields change.

Field	On retry	Source
`transactionIndex` (the withdrawal index)	unchanged	`rebroadcast_expired_withdrawal_documer` touches only status, signHeight, updatedAt, revision
`outputScript` (payout script)	unchanged, immutable after creation	set at `identity_credit_withdrawal/v0/transfo` see immutability note below

Field	On retry	Source
`amount` (payout amount)	unchanged	set at creation `transformer.rs:170`, never re-set
`coreFeePerByte` (and thus the Core fee)	unchanged	set at creation `transformer.rs:171`; fee derived as fixed-size × rate at `document_try_into_asset_unlock_base_tr`
`transactionSignHeight`	changes to current Core height	`rebroadcast.../v1/mod.rs:74-77`
`quorumHash`	may change to the current quorum	attached fresh at dequeue `dequeue_and_build_unsigned_withdrawal_`
`quorumSig`	new signature over the new height	built at `unsigned_withdrawal_txs/v0/mod.rs:162-` applied to the payload at `append_signatures_and_broadcast_withdr`

The re-queued transaction bytes for the base payload (index, output, amount, fee) are byte-identical across retries, because rebroadcast moves the same untied bytes back to the signing queue by index (`MoveBroadcastedWithdrawalTransactionsBackToQueueForResigning`, `drive_op_batch/withdrawals.rs:177-218`). Only the request height and quorum are reattached before each re-sign. This is a re-sign of the same logical withdrawal at a new height, not a new withdrawal.

Current 1:1 model, and the batched end state. The current implementation emits exactly one withdrawal per asset unlock transaction, one index and one output, `build_untied_withdrawal_transactions_from_d` maps each document to its own `transaction_index = start_index + i` and `document_try_into_asset_unlock` emits a single `TxOut` (`output: vec![tx_out]`). This is a deliberate simplification for the accelerated Platform release, and Core's 32-output capacity per asset unlock transaction (`MAXIMUM_WITHDRAWALS = 32`, `assetlocktx.h:76`) is unused. Per feedback from the Platform team, the end-state design batches several concurrent withdrawals into one transaction to reduce fees and space. This does not break the retry-payout invariant, but it extends it, an index then maps to a fixed set of outputs rather than one, and the property a receiver needs is that a batch, once formed, is re-signed as an immutable unit under a stable index, with no silent re-batching of a withdrawal under a different index on retry. Under that rule the reasoning above carries over, credit on mempool arrival, deduplicate by index, and locate the recipient's own output in the batch. If a withdrawal could move between batches on retry, the binding must follow the individual withdrawal, a stronger requirement the spec should state.

The payout immutability rests on two enforcement points, not one. The withdrawal document can only be created by the system withdrawal transition, since the contract sets `creationRestrictionMode: 2` (`withdrawals-documents.json:2-4`), which maps to `NoCreationAllowed` for ordinary identities (`restricted_creation/mod.rs:29-34`, enforced at `document_create_transition_action/advanced_structure_v0/mod.rs:81-104`). And any replacement of an existing withdrawal document is rejected outright by the withdrawal data-trigger binding (`data_triggers/bindings/list/v0/mod.rs:91-97`, `reject/v0/mod.rs:25-40`). So neither a user nor a later transition can rewrite `outputScript` or `amount` after creation.

The signing architecture reinforces this. The DIP-7 signing Request ID is `SHA256(SHA256("plwdtx",`

`index))`, a pure function of the index (Core side `assetlocktx.cpp:132`, Platform side `unsigned_withdrawal_txs/v0/mod.rs:173-189`), while the signed message is the full transaction. Within one quorum, the standard signing-conflict protection already prevents two different payouts under one index. Across quorums, the invariant is upheld not by the signing layer but by Platform reusing the immutable document, which is what this audit confirms.

Why crediting on mempool arrival is safe

Three facts compose into receiver safety.

- A withdrawal in a node's mempool has a verified quorum signature. `CheckAssetUnlockTx` ends by returning `VerifySig(...)` (`assetlocktx.cpp:183`), and it runs on the mempool-acceptance path (`validation.cpp:988` via `CheckSpecialTx`). What mempool acceptance skips is only the duplicate-index check and the credit-pool withdrawal-limit check, both deferred to mining and block validation. The signature itself is checked.
- The payout for an index cannot vary, per the table above.
- An index can settle at most once on the active chain. Once a transaction for an index is mined, the index enters the credit pool's persistent range-set, and any later block reusing it on that chain is rejected (`creditpool.cpp:171-174`, "failed-getcreditpool-index-duplicated"). The index set is built from the active chain's credit-pool state (`specialtxman.cpp:668-684`, `creditpool.cpp:148-173`), so this is a same-chain guarantee, ChainLock is the boundary that makes it permanent. Two same-index transactions may sit in the mempool together, but the miner admits only one (`miner.cpp:548-561`) and the other simply expires.

So a receiver whose own fully validating Core node holds the withdrawal in its mempool, and which deduplicates credits by withdrawal index rather than by transaction id, knows exactly what payout will settle for that index. Deduplication by index is required, not optional, because a re-sign changes the txid while paying the same recipient.

One precision on where this guarantee lives. The payout-invariance is enforced at the Platform layer, by the immutable withdrawal document plus the honest-quorum threshold that signs it, not by a Core consensus rule. Core does not independently reject a second asset unlock that reused an index with a different output or amount before mining, two same-index transactions can coexist in the mempool, and the one-mine-per-index rule only fires at block validation. Under the honest implementation this is airtight, since Platform always rebuilds the same document and producing a divergent payout for one index would require a colluding quorum majority, which is the same trust assumption the withdrawal already rests on. But it means the guarantee is currently an implementation property, not a consensus invariant, which is exactly why writing it into Asset Locks v2 matters, and why a defensive Core-side check rejecting a same-index payout conflict would be the way to harden it further (that check is the Path B territory of the scoping document).

Two honest boundaries separate payout-certainty from settlement-finality, and a receiver should treat them as two tiers. Payout-certainty is immediate, from the immutable payout and the one-mine-per-index rule the receiver knows what will settle the moment it sees a signed unlock. Settlement-finality is ChainLock, not mere mining. A block can reorg before it is ChainLocked, which would un-mine the transaction, and because the payout is immutable the same withdrawal would simply re-mine, but a receiver that credited on mine-not-yet-chainlocked carries the ordinary short reorg risk until the ChainLock lands. This is

why Platform itself marks a withdrawal COMPLETE only on `Chainlocked`, not on `mined` (`update_broadcasted_withdrawal_statuses/v0/mod.rs:113-132`), and why the status RPC distinguishes `mempooled`, `mined`, and `chainlocked` (`rpc/rawtransaction.cpp:805-826`). Separately, the credit-pool withdrawal limit (2000 DASH over a 576-block window under the current withdrawals deployment, rising to 4000 under the V24 limit, which is `NEVER_ACTIVE` on mainnet at this commit, `creditpool.h:120-123`, `creditpool.cpp:189-192`, `chainparams.cpp:212-214`) can delay when a withdrawal mines without ever changing what it pays. None of these is a double-spend or wrong-payout exposure, they are the same liquidity and reorg-risk decisions InstantSend crediting already involves, and a cautious integrator keys final settlement to the `chainlocked` status.

Core self-protects independently of Platform, and that shapes what instant acceptance can safely mean. The withdrawal limit is not merely a delay, it is a deliberate circuit breaker against a compromised or buggy Platform trying to unlock too much, and it is enforced at mining and block validation rather than at mempool acceptance. `ProcessLockUnlockTransaction` runs the limit check in the miner (`miner.cpp:561`) and at block validation, while mempool `CheckSpecialTx` verifies only signature, format, and expiry. So a signature-valid unlock can sit in the mempool yet be declined at mining until the 576-block window frees headroom, and a buggy Platform could sign an unlock the limit will never admit. That draws a clean line between two things the design must keep separate. Instant acceptance, a receiver crediting its own books on a signature-valid mempool unlock, is a receiver-side risk decision that trusts the Platform quorum and treats the limit as a possible delay, and it takes nothing away from Core. Chaining, building a child transaction that spends an unmined unlock's outputs, is different and unsafe, because the unlock's txid is not final until mined (a re-sign changes it) and the unlock is not guaranteed to mine at all, so children would be invalidated. The spec ask below is only for the first. It does not seek to weaken the withdrawal limit or to make unlocks chainable before they are mined, both of which are Core's protection when the happy path fails.

Two edge cases the spec should close

Both fall on the withdrawing user, not the receiver, and both come from the same root, Platform debits the identity's balance up front at document creation (`identity_credit_withdrawal_transition.rs:33-54`) and there is no abandonment state and no re-credit path anywhere in the withdrawal code.

1. Stuck-withdrawal has no refund. EXPIRED is never terminal, it always cycles back to BROADCASTED, one per block (`retry_signing_expired_withdrawal_documents_per_block_limit = 1`). This retry cycling is good for the receiver and matches the intent of the strong-liveness form the companion request asks for, but note the precise guarantee, Platform re-signs indefinitely, which is not the same as guaranteed eventual mining, since the re-signed transaction carries the same fixed fee (finding 2). If a withdrawal can genuinely never be mined, the credits stay debited with no automatic return. A grep of the withdrawals and Drive paths finds no `AddToIdentityBalance` refund.

2. The fee never rises on retry, yet a low fee is a documented reason for non-mining. `coreFeePerByte` is fixed at creation and identical across all retries. Asset-unlock fee validation at block-consensus time only checks that the fee is nonzero and in range (`GetAssetUnlockFee`, `assetlocktx.cpp:192-197`), so the mining obstacle is Core mempool relay policy, which can reject a transaction paying below the node's minimum fee

(`validation.cpp:951-960`). If every relevant Core node's mempool rejects the fixed fee, the Platform broadcast path stalls, and combined with finding 1 the withdrawal is debited and never refunded. DIP-27 itself names "Core fees were too low" as a reason an unlock may not mine. The companion request already permits the fee to change across signings while keeping the payout fixed, which is exactly the lever that would close this, so the recommendation is to make fee-bump-on-retry an explicit allowed behavior. It is receiver-safe because the fee is carried in a separate payload field and is debited from the credit pool on top of the payout, never taken from it (Core `creditpool.cpp:30-45`, `toUnlock = fee + sum(vout)`, Platform `document_try_into_asset_unlock_base_transaction_info/v0/mod.rs:29-42`), so bumping it does not change what the recipient receives and does not disturb dedup-by-index.

What this means for the companion spec request

The Asset Locks v2 retry-payout request asks for four things. Against the implementation:

- Payout fixed per index across signings, already true, immutable document fields. The request becomes a write-down of existing behavior as a normative guarantee, which is the cheap and correct outcome.
- Fee never taken from the payout, already true, separate fee field debited from the pool.
- Retry-until-mined liveness, already the implemented behavior, EXPIRED cycles indefinitely.
- Receiver may treat a mempool unlock as payout-final and deduplicate by index, supported by the code, and worth stating so integrators have one paragraph to cite.

The one refinement the audit adds to the request, allow and specify fee-bump-on-retry, and pair the retry-until-mined guarantee with a defined resolution for a withdrawal that is genuinely unmineable, since today that path debits the user with no refund. These protect the withdrawing user and do not weaken the receiver guarantee.

Sub-questions from the audit scope

- How are fees assigned to these input-less transactions? The fee is an explicit `fee` field in the payload (Core `assetlocktx.h:79-84`), validated by `GetAssetUnlockFee` (`assetlocktx.cpp:186-198`), and debited from the credit pool since there are no inputs.
- When does an unlock leave the mempool? At its expiry height, `requestedHeight + 48` (`HEIGHT_DIFF_EXPIRING = 48`, `assetlocktx.h:150`), enforced by `removeExpiredAssetUnlock` on each connected block (`txmempool.cpp:1132-1146`, `validation.cpp:3026`). Two same-index unlocks can coexist in the mempool but only one can ever mine.
- Is there a status query for receivers? Yes, the `getassetunlockstatuses` RPC returns `chainlocked | mined | mempooled | unknown` per index (`rpc/rawtransaction.cpp:709,805-826`), which is the surface an integrator would poll and the natural place to expose an abandoned state if one is ever defined.